Veritas

Evidence-Guided Hallucination Verification & Self-Correction Web Service

By: Faria Ahmed

A Machine Learning Model Deployment Project

Deploying NLP and NLI Models as a Production Web Service via FastAPI

Table of Contents

1. Introduction

1.1 Overview of the Project

Large Language Models (LLMs) have become a core component of modern AI-driven applications, but they remain prone to generating hallucinations: confident, fluent statements that are factually incorrect or unsupported by evidence. This poses a significant challenge for any application that depends on the reliability of generated text, including chatbots, research assistants, customer support systems, and knowledge tools.

Veritas is a full-stack machine learning web service that addresses this problem directly. Rather than simply generating text and returning it to the user, Veritas wraps the generation step inside an automated verification pipeline. Every claim produced by the underlying LLM is independently checked against a curated knowledge base using a dedicated Natural Language Inference (NLI) model, scored for factual reliability, and where necessary automatically corrected before the final response is returned to the user.

Deploying a machine learning model as a web service typically involves training or selecting a model, building a web application to serve predictions through API endpoints, testing the service locally, and deploying it to a production environment so that other applications can integrate with it over the internet. Veritas follows exactly this lifecycle, but applies it to a multi-model pipeline rather than a single model: it deploys a generative LLM, a sentence-embedding model, and an NLI classification model together behind one unified API, served using FastAPI instead of Flask, with the entire stack containerised using Docker for production deployment.

1.2 Objective

The project was designed around the following objectives:

- Build an automated, end-to-end pipeline that detects unsupported or fabricated factual claims in LLM-generated text.
- Combine dense retrieval (semantic search) with Natural Language Inference verification to determine whether each claim is supported by evidence.
- Generate constructive, evidence-grounded critiques for any claim found to be unsupported.
- Implement an iterative self-correction loop that revises only the flagged claims, bounded by a maximum iteration count.
- Deploy the complete pipeline as a production-ready, containerised web service exposing REST API endpoints.
- Log every pipeline execution with structured, queryable experiment data for evaluation and analysis.

2. Methodology

2.1 Tools and Technology Stack

Veritas deploys three distinct machine learning models behind a single web service: a generative language model for response drafting, a sentence-embedding model for semantic retrieval, and a Natural Language Inference model for factual verification. The framework chosen to serve these models as a web API was FastAPI, selected specifically for its native asynchronous request handling, automatic interactive API documentation, and built-in request/response validation, all of which are valuable when orchestrating multiple sequential model calls per request.

Layer	Technology	Purpose
Web Framework	FastAPI + Uvicorn	Serves the ML pipeline as an asynchronous REST API
LLM Serving	Ollama (local inference server)	Hosts and serves the generative language model
Generative Model	Gemma 3 (via Ollama)	Produces the initial draft response to user queries
Embedding Model	Sentence-Transformers (BAAI/bge-small-en-v1.5)	Converts claims and evidence into dense vectors
Vector Database	FAISS (IndexFlatIP)	Performs fast cosine-similarity search over the corpus
Verification Model	DeBERTa-v3 NLI (cross-encoder)	Classifies claim-evidence pairs as entailment, neutral, or contradiction
Knowledge Base	Curated Wikipedia snapshot	Source corpus for evidence retrieval
Data Validation	Pydantic / Pydantic Settings	Request validation and typed configuration management
Experiment Logging	Structured JSON files	Lightweight, file-based logging of every pipeline run
Containerisation	Docker + Docker Compose	Packages the API and Ollama service for deployment
Testing	Pytest + pytest-asyncio	Unit testing of pipeline modules in isolation

2.2 Methodology and Implementation Steps

The project followed the standard lifecycle for deploying a machine learning model as a web service:model preparation, API development, local testing, and production deployment:adapted for a multi-model pipeline rather than a single model.

Step 1 : Model and Provider Preparation

Rather than hardcoding any single model library, a provider abstraction layer was built first. BaseLLMProvider and BaseEmbeddingProvider interfaces were defined so that the generative model and the embedding model could each be swapped for an alternative implementation without modifying any downstream pipeline code. The Ollama LLM provider and the Sentence-Transformer embedding provider were implemented against these interfaces.

Step 2 : Knowledge Base Construction

A curated subset of Wikipedia articles was streamed from Hugging Face Datasets, split into overlapping word-based chunks, embedded using the sentence-embedding model, and indexed into a FAISS IndexFlatIP structure for exact cosine-similarity search. This indexing step is deliberately separated from the online API: it is run once, offline, before the web service starts.

Step 3 : Building the Verification Pipeline

The core ML logic was implemented as seven independent, testable pipeline modules:

- Response Generator : produces the initial answer from the LLM.
- Claim Extractor : decomposes the response into atomic, independently verifiable factual claims using a fixed structured-output prompt.
- Evidence Retriever : embeds each claim and retrieves the top-k most similar passages from the FAISS index.
- NLI Verifier : runs the DeBERTa-v3 cross-encoder on each claim-evidence pair, returning entailment, neutral, contradiction, or uncertain (when confidence is below threshold).
- Hallucination Scorer : aggregates NLI results into a structured hallucination score and supporting metrics.
- Critique Generator : produces a specific, evidence-grounded correction instruction for each unsupported claim.
- Self-Correction Engine : regenerates the response using the critique, re-verifies it, and repeats up to a maximum of three iterations, always retaining the best-scoring response seen.

Step 4 : Building the Web Service Layer

A FastAPI application was built to expose this pipeline as a set of REST endpoints, with Pydantic models enforcing strict request and response validation. An application lifespan handler loads the FAISS index and warms up both the embedding model and the NLI model at startup, ensuring the first real request is not penalised by model loading latency.

Step 5 : Local Testing

The service was tested locally using two complementary approaches: automated unit tests using Pytest with mocked providers (covering claim parsing, scoring logic, NLI threshold decisions, and the correction loop), and manual interactive testing through FastAPI's automatically generated Swagger UI at the /docs endpoint, which allowed every endpoint to be executed and inspected directly in the browser without writing any client code.

Step 6 : Experiment Logging

Every full pipeline execution is logged to a structured JSON file, including the original response, every extracted claim, its NLI label and confidence, the generated critique, the corrected response, and the complete iteration history. A lightweight JSONL index allows fast retrieval of past runs, and an aggregate metrics endpoint summarises performance across all logged runs.

Step 7 : Containerisation and Deployment

The complete service was containerised using a multi-stage Dockerfile, and a Docker Compose configuration was written to orchestrate the API container alongside an Ollama container, including an automated model-pull step so the required LLM is available before the API becomes healthy. This produces a deployment-ready stack that can be started with a single command.

3. System Architecture

Veritas is architected as a layered system: a web service layer exposes REST endpoints, an orchestration layer coordinates the seven-stage pipeline, a provider abstraction layer isolates the ML models from the rest of the application, and a persistence layer handles corpus storage and experiment logging.

3.1 Pipeline Flow

Stage	Component	Input → Output
1	Response Generator	User query → Draft LLM response
2	Claim Extractor	Draft response → List of atomic claims
3	Evidence Retriever	Claims → Top-k evidence chunks per claim (FAISS)
4	NLI Verifier	Claim + evidence → Entailment / Neutral / Contradiction / Uncertain
5	Hallucination Scorer	NLI labels → Hallucination score + 6 metrics
6	Critique Generator	Unsupported claims → Targeted correction instructions
7	Self-Correction Engine	Critique + original response → Corrected, re-verified response

3.2 Web Service Architecture

The web service is organised into clearly separated layers so that the machine learning logic remains independent of the API framework:

- API Layer (FastAPI) : defines REST endpoints, validates requests with Pydantic schemas, and serializes responses.
- Orchestration Layer : the VeritasPipeline class wires all seven pipeline stages together and owns the correction loop.
- Provider Layer : BaseLLMProvider and BaseEmbeddingProvider abstractions decouple the pipeline from specific model implementations.
- Persistence Layer : the FAISS corpus index (offline-built, loaded read-only at runtime) and the JSON-based experiment logger.

3.3 Exposed API Endpoints

Method	Endpoint	Description
POST	/pipeline	Runs the complete end-to-end verification and correction pipeline
POST	/generate	Generates a raw LLM response without verification
POST	/verify	Verifies arbitrary text against the knowledge base

POST	/critique	Verifies text and generates correction instructions
POST	/correct	Verifies, critiques, and self-corrects existing text
GET	/health	Reports the health of all loaded models and the corpus index
GET	/metrics	Returns aggregate statistics across all logged pipeline runs
GET	/runs	Lists all previous pipeline executions
GET	/runs/{run_id}	Returns the full detail of a specific execution

3.4 Deployment Architecture

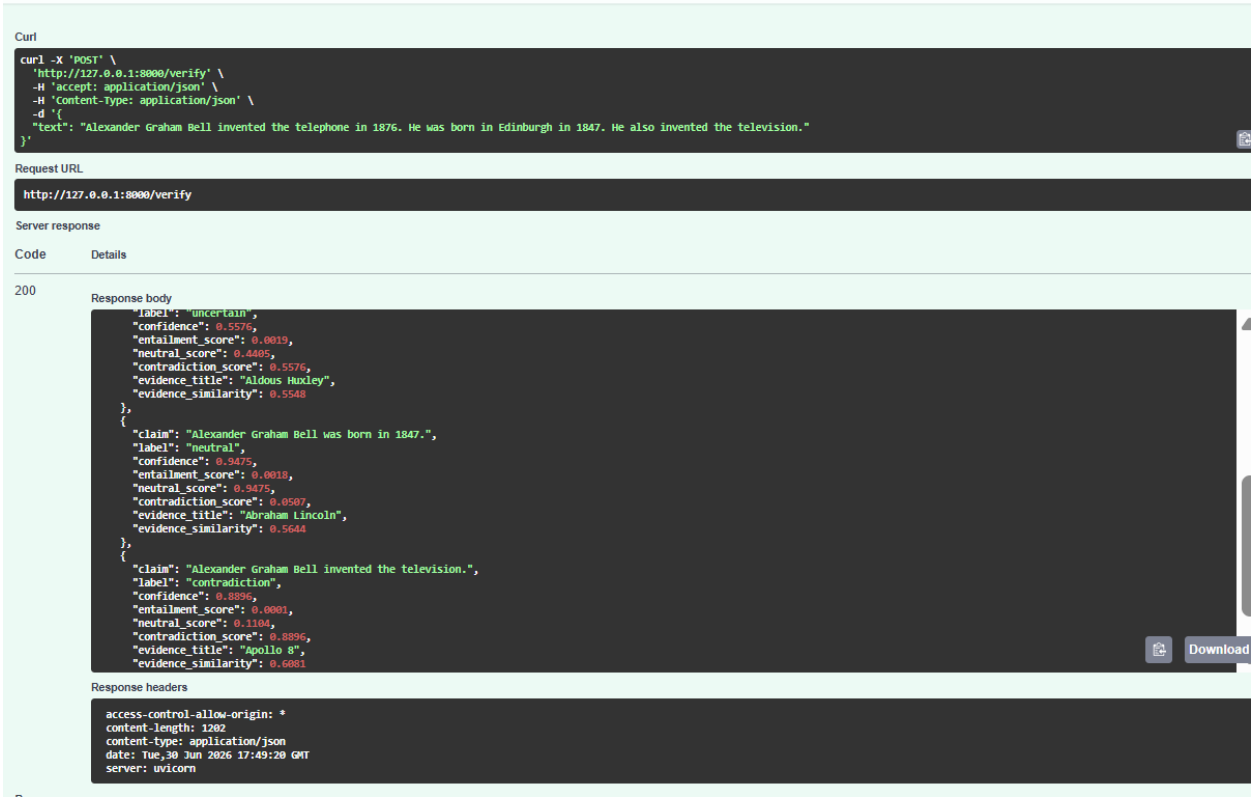
In production, the service is deployed via Docker Compose as two coordinated containers: the Veritas API container, and an Ollama container serving the generative model. The FAISS index is built offline and mounted into the API container as a read-only volume, while experiment logs and the Hugging Face model cache are mounted as persistent writable volumes so that downloaded model weights and logged runs survive container restarts.

4. Results and Observations

4.1 Functional Results

The deployed service was tested against both factual prompts (expected to verify cleanly) and deliberately fabricated or impossible prompts (expected to trigger hallucination detection and correction). The following behaviours were observed consistently across test runs:

- Factual queries with strong corpus coverage (e.g., well-documented historical events) consistently produced low hallucination scores, typically in the 0.0–0.2 range, with the majority of claims labelled as entailment.
- Fabricated or impossible prompts (fictional events, future-dated claims, false premises) reliably produced high initial hallucination scores, frequently above 0.6, correctly flagging the response as substantially unsupported.
- The self-correction loop measurably reduced the hallucination score across iterations in the majority of test cases, with most corrections converging within a single iteration when evidence was available.
- Claims with no matching evidence in the corpus were correctly excluded from the hallucination score denominator, distinguishing a genuine retrieval gap from an active factual contradiction.
- The NLI confidence threshold mechanism correctly downgraded low-confidence classifications to an explicit "uncertain" label rather than forcing an unreliable entailment or contradiction decision.



Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/critique' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
  "text": "Alexander Graham Bell invented the telephone in 1876. He was born in Edinburgh in 1847. He also invented the television."
}'
```

Request URL

```
http://127.0.0.1:8000/critique
```

Server response

Code Details

200

Response body


```
      "label": "uncertain",
      "instruction": "Change “Edinburgh” to “Godalming, Surrey, England” because Aldous Huxley’s biography states he was born in Godalming, Surrey, England in 1894.",
      "evidence_source": "Aldous Huxley"
    },
    {
      "claim": "Alexander Graham Bell was born in 1847.",
      "label": "neutral",
      "instruction": "Change \"1847\" to \"March 31, 1847,\" as evidence from Abraham Lincoln’s biography states Alexander Graham Bell was born on March 31, 1847.",
      "evidence_source": "Abraham Lincoln"
    },
    {
      "claim": "Alexander Graham Bell invented the television.",
      "label": "contradiction",
      "instruction": "Change \"Alexander Graham Bell invented the television” to “The Apollo 8 Christmas Eve transmission won an Emmy Award, recognizing it as one of the first televised events.”",
      "evidence_source": "Apollo 8"
    }
  ],
  "correction_block": "The following 4 claim(s) require correction:\n\n[1] Original claim: \"Alexander Graham Bell invented the telephone.\"\n    Issue: NEUTRAL\n    Instruction: Change “Alexander Graham Bell invented the telephone” to “Telecommunication services in Afghanistan have grown significantly since 2001, with mobile phone subscribers reaching 22 million by 2016.”\n\n[2] Original claim: \"Alexander Graham Bell was born in Edinburgh.\"\n    Issue: UNCERTAIN\n    Instruction: Change “Edinburgh” to “Godalming, Surrey, England” because Aldous Huxley’s biography states he was born in Godalming, Surrey, England in 1894.\n\n[3] Original claim: \"Alexander Graham Bell was born in 1847.\"\n    Issue: NEUTRAL\n    Instruction: Change \"1847\" to \"March 31, 1847,\" as evidence from Abraham Lincoln’s biography states Alexander Graham Bell was born on March 31, 1847.\n\n[4] Original claim: \"Alexander Graham Bell invented the television.\"\n    Issue: CONTRADICTION\n    Instruction: Change \"Alexander Graham Bell invented the television” to “The Apollo 8 Christmas Eve transmission won an Emmy Award, recognizing it as one of the first televised events.”\n"
}
```


Download

Execute Clear

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/pipeline' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
  "query": "Alexander Graham Bell invented the telephone in 1876. He was born in Edinburgh in 1847. He also invented the television."
}
'
```

Request URL

```
http://127.0.0.1:8000/pipeline
```

Server response

Code Details

200

Response body


```
{
  "run_id": "56966a97-bebf-42a3-afd2-f6474d1627d5",
  "query": "Alexander Graham Bell invented the telephone in 1876. He was born in Edinburgh in 1847. He also invented the television.",
  "initial_response": "That statement contains an inaccuracy. Alexander Graham Bell is credited with inventing the first practical telephone, which he demonstrated on March 10, 1876, at a demonstration before the United States Centennial Exposition in Philadelphia. However, Alexander Graham Bell did not invent the television. Philo Farnsworth is widely recognized as the inventor of the first fully functional electronic television system, receiving a patent for his “image dissector tube” in 1927. Bell’s contributions to early telephony were significant, but he did not develop television technology.",
  "final_response": "That statement contains an inaccuracy. Alexander Graham Bell is credited with inventing the first practical telephone, which he demonstrated on March 10, 1876, at a demonstration before the United States Centennial Exposition in Philadelphia. However, Alexander Graham Bell did not invent the television. Philo Farnsworth is widely recognized as the inventor of the first fully functional electronic television system, receiving a patent for his “image dissector tube” in 1927. Bell’s contributions to early telephony were significant, but he did not develop television technology.",
  "claims": [
    "Alexander Graham Bell is credited with inventing the first practical telephone.",
    "Alexander Graham Bell demonstrated the first practical telephone on March 10, 1876.",
    "The demonstration of the first practical telephone was before the United States Centennial Exposition in Philadelphia.",
    "Alexander Graham Bell did not invent the television.",
    "Philo Farnsworth is widely recognized as the inventor of the first fully functional electronic television system.",
    "Philo Farnsworth received a patent for his ‘image dissector tube’ in 1927."
  ],
  "claim_parse_error": "stripped_markdown_fences",
  "initial_metrics": {
    "total_claims": 6,
    "supported_claims": 0,
    "contradicted_claims": 0,
    "neutral_claims": 6,
    "uncertain_claims": 0,
```


Download

Response headers

```
access-control-allow-origin: *
content-length: 6099
content-type: application/json
date: Wed,01 Jul 2026 09:28:02 GMT
server: uvicorn
```

4.2 Performance Observations

Aspect	Observation
Model warm-up	Both the embedding model and NLI model add a one-time startup delay while weights load into memory; the lifespan handler warms these models before the API accepts traffic.
First-request latency	After warm-up, individual pipeline stages (retrieval, NLI inference) execute quickly; total pipeline latency is dominated by LLM generation and correction calls to Ollama.
Correction convergence	Most responses that required correction reached a hallucination score of 0.0 within one to two iterations, validating the three-iteration cap as sufficient for the corpus size tested.
Retrieval coverage	Coverage was strongly dependent on corpus size; claims about topics well represented in the indexed Wikipedia subset were reliably matched, while niche topics outside the curated corpus produced no-evidence results.

4.3 Key Observations

- Decoupling claim extraction from verification proved effective: atomic, single-fact claims produced far cleaner NLI decisions than verifying whole paragraphs at once.
- The provider abstraction layer (BaseLLMProvider / BaseEmbeddingProvider) meant the underlying generative model could be swapped without any change to the verification, scoring, or correction logic:confirming the architecture's modularity goal.
- Returning the best-scoring response across all correction iterations, rather than simply the final iteration, prevented occasional regressions where a later correction introduced a new unsupported claim.
- Excluding no-evidence claims from the hallucination score denominator avoided unfairly penalising the model for gaps in corpus coverage, which is a retrieval limitation rather than a generation hallucination.

5. Conclusion

Veritas demonstrates that the standard lifecycle for deploying a machine learning model as a web service:preparing the model, building an API to serve it, testing locally, and deploying to production:extends naturally to a multi-model verification pipeline. By replacing Flask with FastAPI, the project gained asynchronous request handling and automatic interactive documentation, both of which proved valuable when orchestrating several sequential model calls within a single API request.

The completed system successfully integrates three distinct machine learning models:a generative LLM, a sentence-embedding model, and a Natural Language Inference classifier:behind one cohesive REST API, and adds a genuinely novel capability on top of simple model serving: the service does not just return predictions, it evaluates its own output for factual reliability and autonomously attempts to correct any unsupported claims before responding to the user.

The modular pipeline design, the strict provider abstraction layer, and the bounded, observable self-correction loop together meet the project's objective of producing a production-quality research framework rather than a prototype. The containerised deployment via Docker Compose confirms that the complete multi-model stack can be packaged and run as a single, reproducible service, fulfilling the core goal of making the hallucination verification capability accessible to other applications over the internet through a standard web API.

Future iterations could extend the framework with a larger and more diverse knowledge base, additional swappable model providers (such as OpenAI or Anthropic), evidence reranking, and persistent database-backed experiment tracking:all of which are explicitly supported by the abstraction boundaries already established in the current architecture.